Git / GitHub — 3–4 YOE Scenario-Based Q&A

1. What is Git?
Definition:
Git is a distributed version control system used to track changes in source code and files.
It allows multiple developers to work on the same project without directly overwriting each other's changes.
Git maintains a history of commits, branches, and changes made to the repository.
It also supports collaboration, branching, merging, reverting, and recovery of previous versions.
Simple meaning:
Git ni manam application source code ni version control cheyadaniki use chestham.
Developers separate branches lo work chesi, changes ni commit chesi, later merge cheyyachu.
Previous code versions ni kuda track cheyyachu and required ayithe recover cheyyachu.
Example:
Suppose three developers are working on the same application.
One developer works on the login feature, another works on payment, and another works on notifications.
Each developer can work on a separate branch and later merge the completed changes into the main branch.
Simple flow:
Developer
↓
Git Repository
↓
Branch
↓
Changes
↓
Commit
↓
Merge
Main point:
Git = Version control system used to track, manage, and collaborate on source-code changes.

---

2. What is GitHub?
Definition:
GitHub is a cloud-based platform used to host Git repositories and collaborate on source code.
It provides features such as Pull Requests, code reviews, branch protection, Issues, and Actions.
Teams can use GitHub to manage source code and collaborate on software development.
It can also integrate with CI/CD tools such as Jenkins and other development tools.
Simple meaning:
GitHub ni manam Git repositories ni remote location lo host cheyadaniki and team collaboration kosam use chestham.
Developers code push chesi, Pull Requests create chesi, code reviews perform cheyyachu.
CI/CD pipelines ni GitHub repository changes tho trigger cheyyachu.
Example:
Developer local machine lo code change chestadu.
Aa changes ni Git repository ki commit chesi GitHub repository ki push chestadu.
Team members Pull Request create chesi review chesi, approved ayithe main branch lo merge chestaru.
Simple flow:
Developer
↓
Git
↓
GitHub Repository
↓
Pull Request
↓
Code Review
↓
Main Branch
Main point:
GitHub = Platform used to host Git repositories and collaborate on software development.

---

3. Developer accidentally pushed code directly to the main branch. What will you do?
Definition:
A direct push to the main branch can bypass the normal Pull Request and review process.
First, I check what changes were pushed and whether they affected any CI/CD or Production deployment.
If the change is incorrect, I normally use git revert for a shared branch instead of rewriting its history.
I also check branch protection rules to prevent similar direct pushes in the future.
Simple meaning:
First pushed code correct ah wrong ah check chestha.
Wrong ayithe shared branch kabatti history rewrite cheyakunda revert chestha.
Tarvata CI/CD and Production impact check chesi, branch protection enable chestha.

Example:
Suppose a developer accidentally pushes a configuration change directly to main and the change breaks the application.
I would revert that change, verify the pipeline, and then make sure future changes require a Pull Request and approval.
Simple flow:
Direct Push
↓
Check Changes
↓
Check Impact
↓
Revert if Required
↓
Verify CI/CD
↓
Protect Main Branch
Main point:
Shared main branch → Prefer revert → Verify impact → Enable branch protection.

---

4. Two developers modified the same file and there is a merge conflict. How will you resolve it?
Definition:
A Git merge conflict occurs when Git cannot automatically combine changes from different branches.
This commonly happens when two developers modify the same lines or closely related parts of a file.
Git marks the conflicting sections so the developer can decide which changes should remain.
The conflict must be resolved, tested, and committed before the merge can be completed.
Simple meaning:
Two developers same file lo same area change chesthe Git automatic ga decide cheyyalekapovachu.
Appudu conflict vastundi.
Both developers changes understand chesi correct logic select chesi conflict resolve chestha.
Example:
Suppose Developer A changes the login validation logic and Developer B changes the same login validation lines.
Git marks the conflicting sections, and I review both changes before keeping the correct combined logic.
Simple flow:
Developer A Changes
↓
Developer B Changes
↓
Merge
↓
Conflict
↓
Review Both Changes
↓
Resolve
↓
Test
↓
Commit
Main point:
Conflict → Understand both changes → Resolve correctly → Test → Commit.

---

5. A developer says, "My branch is working, but after merging into main the application is failing." What will you check?
Definition:
A feature branch can work independently but fail after integration with the main branch.
This can happen because of merge conflicts, dependency changes, configuration differences, or interactions with other code.
I compare the feature branch with the latest main branch to identify the integration difference.
I also check CI pipeline logs and application logs to identify the exact failure.
Simple meaning:
Branch individual ga work ayina, main tho merge ayyaka other changes tho conflict or dependency issue ravachu.
First merged changes compare chestha.
Tarvata CI logs and application logs check chesi root cause identify chestha.
Example:
A developer's branch works correctly, but after merging another developer's database configuration change, the application fails.
I compare the changes and identify whether the failure was introduced by the merge or environment configuration.
Simple flow:
Feature Branch
↓
Merge to Main
↓
Application Failure
↓
Compare Changes

↓
Check CI/Logs
↓
Find Root Cause
↓
Fix
Main point:
Works in Branch + Fails in Main → Check integration differences.

---

6. How do you find which commit introduced a Production issue?
Definition:
Git maintains a history of commits, which allows us to identify changes introduced over time.
For a Production issue, I first identify when the problem started and correlate that time with deployments and commits.
I review the relevant commits and changes to determine which modification may have introduced the regression.
For difficult cases, git bisect can perform a binary search through commit history to identify the problematic commit.
Simple meaning:
Production issue eppudu start ayindo first identify chestha.
Aa time ki related commits and deployments check chestha.
Issue exact commit identify cheyyadam difficult ayithe Git bisect use chestha.
Example:
Suppose the application worked correctly yesterday but started failing after today's deployment.
I compare today's deployed commit with the previous known-good commit and investigate the changes between them.
Simple flow:
Production Issue
↓
Identify Start Time
↓
Check Deployment
↓
Check Commits
↓
Compare Changes
↓
Git Bisect if Needed
↓
Problem Commit
Main point:
Production issue → Deployment → Commit history → Compare → Identify bad commit.

---

7. What is the difference between git revert and git reset?
Definition:
git revert creates a new commit that reverses the changes introduced by an earlier commit.
git reset moves the current branch reference to another commit and can modify local history depending on the reset mode.
Revert is generally safer for shared branches because it does not remove existing commits from the shared history.
Reset is more commonly used for local or private work where rewriting history is acceptable.
Simple meaning:
Revert ante previous change ni new commit tho undo cheyyadam.
Reset ante branch pointer ni previous commit ki move cheyyadam.
Shared branch lo generally revert prefer chestha.
Example:
Revert:
A → B → C → Revert C
Reset:
A → B → C
↑
Branch moves back
Simple flow:
Shared Branch
↓
Revert
↓
New Commit
Local Work
↓
Reset
↓
Rewrite Local History
Main point:
Revert = Safely undo with a new commit.
Reset = Move branch history and potentially rewrite it.

---

8. Developer accidentally committed a password to Git. What will you do?
Definition:
A password committed to Git should be treated as an exposed credential.
Removing the password from the latest version alone is not enough because it may still exist in Git history.
I first rotate or revoke the exposed credential to prevent unauthorized use.
Then I remove the secret from the code and clean the repository history when required.
Simple meaning:
Password Git lo commit ayithe first remove cheyadam kanna important credential ni immediately rotate or revoke cheyyadam.
Tarvata code and history nunchi secret remove chestha.
Future lo secrets ni Git lo store cheyakunda secure secrets-management solution use chestha.
Example:
Suppose a database password was accidentally committed and pushed to GitHub.
I immediately change the database password, remove the exposed value, clean the history according to the repository process, and update the application to retrieve the password securely.
Simple flow:
Secret Leak
↓
Rotate / Revoke
↓
Remove Secret
↓
Clean History
↓
Secure Secret Storage
↓
Verify
Main point:
Secret Leak → Rotate first → Remove → Clean history → Secure properly.

---

9. Git repository has become very large. How will you troubleshoot?
Definition:
A Git repository can become large because of binaries, generated files, build artifacts, logs, or large files stored in its history.
First, I identify which files or historical objects are consuming the most space.
I check whether unnecessary generated files or artifacts were committed.
For large files that genuinely need versioning, I consider Git LFS, while build artifacts should generally be stored in an artifact repository.
Simple meaning:
Repository size ekkuva ayithe first large files and old history identify chestha.
Build artifacts, logs, generated files Git lo unnecessarily store chestunnara check chestha.
Required files ki Git LFS and build artifacts ki Nexus or another artifact repository consider chestha.
Example:
Suppose a repository contains large application build files and old binary packages.
I remove inappropriate artifacts from source control and move them to an artifact repository.
Simple flow:
Large Repository
↓
Find Large Files
↓
Check Git History
↓
Remove Unnecessary Artifacts
↓
Git LFS / Artifact Repository
↓
Optimize
Main point:
Git = Source Code
Artifact Repository = Build Artifacts
Git LFS = Large versioned files.

---

10. GitHub webhook is configured, but Jenkins pipeline is not triggering. How will you troubleshoot?
Definition:
A GitHub webhook sends an event to Jenkins when a configured repository activity occurs.
If Jenkins is not triggering, I troubleshoot the complete communication path from GitHub to Jenkins.
I first verify whether GitHub successfully delivered the webhook event.
Then I check Jenkins job configuration, trigger conditions, repository settings, branch filters, and Jenkins logs.
Simple meaning:
First GitHub webhook actually Jenkins ki reach avutunda check chestha.
GitHub side delivery successful ayithe Jenkins configuration and trigger conditions check chestha.
Branch filter and Jenkins logs kuda verify chestha.

Example:
Suppose GitHub shows that the webhook delivery returned successfully, but Jenkins does not start the pipeline.
I would focus on Jenkins job configuration, repository configuration, branch filters, and trigger settings.
Simple flow:
GitHub
↓
Webhook
↓
Jenkins
↓
Job Trigger
↓
Pipeline
Main point:
Webhook issue → Check GitHub delivery → Jenkins trigger → Job configuration → Logs.

---

11. How do you protect the main branch?
Definition:
Main branch protection prevents developers from directly making uncontrolled changes to an important shared branch.
It allows organizations to enforce Pull Requests, reviews, automated checks, and other controls before merging.
It can also restrict direct pushes and force pushes to the branch.
These controls reduce the risk of untested or unauthorized code reaching the main branch.
Simple meaning:
Main branch ki direct push allow cheyakunda Pull Request and code review mandatory chestha.
CI checks pass avvali and required approvals complete avvali.
Force push and unauthorized changes ni restrict chestha.
Example:
A developer wants to add a new feature.
Instead of pushing directly to main, the developer creates a feature branch, opens a Pull Request, waits for review and CI checks, and then merges it.
Simple flow:
Feature Branch
↓
Pull Request
↓
Code Review
↓
CI Checks
↓
Approval
↓
Main
Main point:
Main Branch = Protected + Reviewed + CI validated.

---

12. Production deployment was successful, but you need to know exactly which Git commit is running. How will you identify it?
Definition:
Production traceability means being able to identify the exact source-code revision used by a deployed application.
I associate the Git commit SHA or release version with the build artifact.
The same identifier is then carried through the CI/CD pipeline into the deployment.
This allows us to identify exactly which source revision is running in Production.
Simple meaning:
Production lo application run avutundi ante adi ye Git commit nunchi build ayindo clearly teliyali.
Anduke build artifact ki Git commit SHA or unique version attach chestha.
Deployment time lo same identifier ni track chestha.
Example:
Suppose Git commit 8f32a1 was used to build the application.
The resulting artifact or Docker image is associated with that commit, allowing the Production deployment to be traced back to the exact source revision.
Simple flow:
Git Commit
↓
CI Build
↓
Versioned Artifact
↓
Deployment
↓
Production

↓
Trace to Commit
Main point:
Git SHA → Build Artifact → Deployment → Production Traceability.

---

13. Why is using the latest Docker tag risky in Production?
Definition:
The latest Docker tag is mutable, which means it can point to different image versions over time.
A deployment using latest may therefore run a different image later without the deployment configuration changing.
This makes troubleshooting, auditing, and rollback less predictable.
For Production, I prefer immutable version tags or Git commit-based image tags.
Simple meaning:
latest same name unna different image versions ki point avvachu.
Kabatti Production lo exact ga ye version deploy ayindo confusion ravachu.
Unique version or Git SHA tags use chestha.
Example:
Today:
latest → v1.0
Later:
latest → v1.1
Production configuration same unna actual image different ga undachu.
Simple flow:
Code
↓
Git SHA
↓
Docker Build
↓
Unique Image Tag
↓
Registry
↓
Production
Main point:
Production → Prefer immutable/versioned image tags instead of relying on latest.

---

14. A developer wants to merge a feature, but CI is failing. Should you merge it?
Definition:
A failing CI pipeline indicates that one or more automated validation checks have not passed.
Normally, I do not merge the Pull Request until the failure is understood.
I first determine whether the failure is caused by application code, configuration, dependency changes, or CI infrastructure.
If the failure is unrelated infrastructure noise, I follow the organization's approved process rather than blindly bypassing the check.
Simple meaning:
CI fail ayithe direct ga merge cheyyanu.
First failure reason identify chestha.
Code issue ayithe fix cheyyali; infrastructure issue ayithe investigate and approved process follow cheyyali.
Example:
Suppose unit tests are failing because of a code change.
I would ask the developer to fix the issue and rerun CI before approving the merge.
Simple flow:
Pull Request
↓
CI
↓
Failed
↓
Find Cause
↓
Fix / Resolve Infrastructure Issue
↓
CI Passed
↓
Merge
Main point:
CI Failed → Understand the failure → Fix or properly resolve → Then merge.

---

15. How do you handle a hotfix for a Production issue?
Definition:
A Production hotfix is a controlled change created specifically to resolve an urgent Production issue.
The hotfix should be based on the appropriate Production or release baseline.

After implementing the fix, I run the required validation and deploy it through the approved CI/CD process.
The same fix must also be merged back into the main development line so that it is not lost in future releases.
Simple meaning:
Production lo critical issue vaste normal feature development tho mix cheyakunda controlled hotfix create chestha.
Fix ni test chesi CI/CD through Production ki deploy chestha.
Tarvata same fix ni main development branch ki merge chestha.
Example:
Suppose a Production application has a critical authentication bug.
I create a hotfix from the appropriate release baseline, apply the fix, validate it, deploy it, and then merge the fix back into the main development branch.
Simple flow:
Production Issue
↓
Hotfix Branch
↓
Fix
↓
Test
↓
CI/CD
↓
Production
↓
Merge Back
Main point:
Production Issue → Controlled Hotfix → Test → Deploy → Merge Back.

Git / GitHub — PART-2
16. Developer accidentally deleted a branch. Can you recover it?
Definition:
A deleted Git branch can often be recovered if the commits that belonged to the branch are still reachable or recorded in Git references.
The important step is to identify the last known commit of the deleted branch.
Git reflog can often help identify previous branch or HEAD positions in a local repository.
Once the correct commit is identified, a new branch can be created from that commit.
Simple meaning:
Branch delete ayina ventane commits necessarily disappear avvavu.
First last known commit identify chestha.
Aa commit dorikithe new branch create chesi work recover cheyyachu.
Example:
Suppose a developer accidentally deletes a feature branch after completing several commits.
If the last commit SHA can be identified, I can recreate the branch from that commit.
Simple flow:
Deleted Branch
↓
Find Last Commit
↓
Verify Commit
↓
Recreate Branch
↓
Verify Changes
Main point:
Deleted Branch → Find Commit → Recreate Branch.

---

17. What is the difference between git merge and git rebase?
Definition:
Git merge combines the histories of two branches and may create a merge commit.
Git rebase moves or reapplies commits from one branch onto a new base commit.
Merge preserves the existing branch history, while rebase can create a more linear history.
Because rebase rewrites commit history, it should be used carefully on commits that are already shared.
Simple meaning:
Merge ante two branches ni combine chestham and existing history preserve chestham.
Rebase ante current branch commits ni new base paina replay chestham.
Shared branch history ni unnecessarily rewrite cheyakunda careful ga use cheyyali.
Example:
Merge:
Feature Branch
↓
Merge

↓
Main
Rebase:
Feature Commits
↓
New Main Base
↓
Reapplied Commits
Simple flow:
Merge → Combine Histories → Preserve History
Rebase → New Base → Replay Commits → Linear History
Main point:
Merge = Combine histories.
Rebase = Reapply commits on a new base and potentially rewrite history.

---

18. A developer rebased a shared branch and pushed with force. What will you do?
Definition:
Rebasing a shared branch changes its commit history and force-pushing can replace the remote branch history.
This can cause problems for developers who already based their work on the previous history.
First, I assess what changed and identify whether any commits were lost or made difficult to reach.
If necessary, I recover the previous branch state and then establish safer branch protection rules.
Simple meaning:
Shared branch ni rebase chesi force push chesthe other developers work impact avvachu.
First previous history identify chestha.
Required commits recover chesi branch ni safe state ki restore chestha.
Example:
Suppose a shared development branch contained several developers' commits and someone force-pushed a rebased version.
I compare the old and new history, recover missing commits if necessary, and prevent future force pushes through branch protection.
Simple flow:
Force Push
↓
Check Impact
↓
Compare History
↓
Recover Missing Commits
↓
Restore
↓
Protect Branch
Main point:
Shared Branch + Force Push → Assess → Recover → Restore → Prevent.

---

19. How do you handle a bad commit that was pushed to a shared branch?
Definition:
A bad commit on a shared branch should normally be removed without rewriting the branch history.
I first identify the exact commit and understand whether later commits depend on it.
For a shared branch, I generally use a revert so that a new commit reverses the unwanted change.
After the revert, I validate the application and then introduce the corrected change through the normal review process.
Simple meaning:
Shared branch lo wrong commit push ayithe history rewrite cheyakunda revert prefer chestha.
Revert tarvata CI and application verify chestha.
Correct fix ni separate reviewed change ga merge chestha.
Example:
Suppose a configuration commit breaks the application after being pushed to main.
I revert the problematic commit, verify the application, and later merge the corrected configuration through a Pull Request.
Simple flow:
Bad Commit
↓
Identify
↓
Revert
↓
CI / Test
↓
Correct Fix
↓
Pull Request
Main point:
Shared Branch → Revert bad commit → Verify → Correct properly.

20. Git pull is showing conflicts. How will you resolve them?
Definition:
A Git pull conflict occurs when local and remote changes cannot be automatically combined.
Git identifies the files and sections where the changes overlap.
I review both versions and determine the correct application logic before resolving the conflict.
After resolving the conflict, I test the result and complete the merge or rebase operation.
Simple meaning:
Remote changes and local changes same area lo unte Git automatic ga combine cheyyalekapovachu.
Conflict vachaka both changes understand chesi correct logic maintain chestha.
Tarvata test chesi merge or rebase complete chestha.
Example:
Suppose I changed an API configuration locally while another developer changed the same configuration remotely.
Git reports a conflict, so I review both changes and combine the correct configuration.
Simple flow:
Local Changes
+
Remote Changes
↓
Conflict
↓
Review Both
↓
Resolve
↓
Test
↓
Complete Pull
Main point:
Conflict → Review both sides → Resolve correctly → Test → Complete.

---

21. A developer committed changes locally but has not pushed them. They want to remove the commit. What will you do?
Definition:
A local commit that has not been pushed can usually be modified without affecting other developers.
The appropriate Git operation depends on whether the developer wants to keep the file changes or remove them completely.
If the commit needs to be modified, the developer can amend or rewrite the local history.
Since the commit is not shared, rewriting the local history is generally safer.
Simple meaning:
Commit local lo matrame unte other developers ki impact undadu.
Changes keep cheyyala, modify cheyyala, completely remove cheyyala ane requirement batti operation select chestha.
Example:
Suppose a developer committed code but forgot to include one required file.
Instead of creating unnecessary history, the developer can modify the local commit before pushing it.
Simple flow:
Local Commit
↓
Check Requirement
↓
Modify / Remove
↓
Test
↓
Push
Main point:
Local unpushed commit → History can be safely modified based on requirement.

---

22. How do you undo uncommitted changes in Git?
Definition:
Uncommitted changes are modifications currently present in the working tree but not yet stored in a commit.
Before removing them, I verify whether the changes are still required because discarding them can cause data loss.
If the changes are not needed, I safely discard the appropriate working-tree changes.
If the changes may be needed later, I preserve them temporarily instead of deleting them.
Simple meaning:
Commit cheyyani changes unte first avi important ah kadha check chestha.
Need lekapothe discard chestha.
Later use avvachu ante temporary ga preserve chestha.
Example:
Suppose a developer made experimental code changes that are no longer required.
After confirming the changes are not needed, I restore the working tree to the required state.
Simple flow:

Uncommitted Changes
↓
Check Importance
↓
Keep or Discard
↓
Clean Working Tree
Main point:
Uncommitted Changes → Check first → Preserve or safely discard.

---

23. How do you temporarily save unfinished Git changes?
Definition:
Git stash provides a way to temporarily save working-tree changes without creating a normal project commit.
It is useful when a developer needs a clean working tree to switch branches or work on another task.
The saved changes can later be restored when the developer returns to the original work.
Stash is intended for temporary work and should not be treated as a permanent backup mechanism.
Simple meaning:
Current work complete kakapoyina urgent ga another branch ki switch avvali ante stash use chestha.
Changes temporary ga save avutayi and working tree clean avutundi.
Later same changes restore chesi continue cheyyachu.
Example:
Suppose a developer is working on a feature but receives an urgent Production task.
The unfinished feature changes can be temporarily stashed, the hotfix can be handled, and the feature work can later be restored.
Simple flow:
Unfinished Work
↓
Stash
↓
Clean Working Tree
↓
Switch Branch
↓
Complete Urgent Work
↓
Restore Stashed Work
Main point:
Git Stash = Temporarily save unfinished working changes.

---

24. Git says the branch is behind the remote branch. What will you do?
Definition:
A branch is behind when the remote branch contains commits that are not present in the local branch.
This normally happens when another developer has pushed changes to the same remote branch.
I first check my local working state and retrieve the latest remote information.
Then I update the local branch using the team's approved integration strategy and resolve conflicts if necessary.
Simple meaning:
Remote branch lo new commits unnayi kani local branch lo avi levu ante local branch behind lo undi.
First latest remote changes fetch chestha.
Tarvata local branch update chesi conflicts unte resolve chestha.
Example:
Suppose another developer pushes three commits to the shared development branch while my local copy is still at the older commit.
My local branch is now behind the remote branch and needs to be updated before continuing.
Simple flow:
Local Branch
↓
Remote Has New Commits
↓
Fetch
↓
Compare
↓
Update Local Branch
↓
Resolve Conflicts if Needed
Main point:
Behind Remote → Fetch latest changes → Update local branch.

---

25. Git says the branch has diverged. What does it mean and how will you fix it?
Definition:
A branch has diverged when both the local and remote branches contain different commits that are not present on the other side.
This means the local branch has its own commits while the remote branch has additional commits.

I first inspect the commit history to understand both sets of changes.
Then I integrate them using the team's approved merge or rebase strategy and resolve any conflicts.
Simple meaning:
Local lo kuda commits unnayi, remote lo kuda different commits unnayi ante branches diverged ayyayi.
First both histories compare chestha.
Tarvata team strategy batti merge or rebase chesi conflicts resolve chestha.
Example:
Local:
A → B → C
Remote:
A → B → D
Both sides have different commits after B, so the histories have diverged.
Simple flow:
Local Commits
+
Remote Commits
↓
Diverged
↓
Compare History
↓
Merge / Rebase
↓
Resolve Conflicts
↓
Test
Main point:
Diverged = Local and remote both have different commits.

---

26. How do you remove a file containing secrets from Git history?
Definition:
Removing a secret from the latest version of a repository does not automatically remove it from previous Git commits.
If the secret was exposed, I first rotate or revoke the credential because the old value must be considered compromised.
Then I remove the secret from the current code and clean the repository history using the organization's approved process.
I also verify that the secret is no longer exposed and implement controls to prevent future secret commits.
Simple meaning:
Secret current file nunchi remove chesina old Git history lo still undachu.
Kabatti first credential rotate chestha.
Tarvata repository history clean chesi future lo secret commits prevent chestha.
Example:
Suppose an API token was committed several months ago and exists in multiple historical commits.
I rotate the token first, clean the affected history, and update the application to use secure secret management.
Simple flow:
Secret Found in History
↓
Rotate / Revoke
↓
Remove from Current Code
↓
Clean History
↓
Verify
↓
Prevent Future Exposure
Main point:
Secret in Git history → Rotate first → Clean history → Secure secret management.

---

27. How do you use Git tags for Production releases?
Definition:
Git tags are references used to mark specific commits in a repository.
They are commonly used to identify release versions and important points in the project's history.
A Production release can be associated with a specific tag so that the exact source revision is known.
This improves traceability, release management, and rollback planning.
Simple meaning:
Production release ki exact code version identify cheyyadaniki Git tags use chestha.
For example, release ki specific version tag create chestha.
Later aa tag chusi exact source revision identify cheyyachu.
Example:
Suppose a tested Production release is approved as version v2.4.0.
The release is tagged with v2.4.0, allowing the team to identify the exact commit associated with that release.
Simple flow:

Code
↓
Test
↓
Approved Commit
↓
Git Tag
↓
Build
↓
Production Release
Main point:
Git Tag = Stable reference to a specific commit or release.

---

28. A GitHub Pull Request was merged accidentally. What will you do?
Definition:
An accidentally merged Pull Request can introduce unreviewed or incorrect changes into a shared branch.
First, I identify what was merged and whether it triggered any CI/CD deployment.
If the change is incorrect, I normally create a controlled revert rather than rewriting shared branch history.
After restoring the correct state, I investigate why the Pull Request was allowed to merge and strengthen the required controls.
Simple meaning:
PR accidental ga merge ayithe first impact check chestha.
Production or other environment ki deploy ayinda check chestha.
Wrong change ayithe shared history rewrite cheyakunda revert chestha.
Example:
Suppose a Pull Request was merged without the required approval and immediately triggered a Production deployment.
I assess the Production impact, revert the incorrect change if required, and review branch protection and approval settings.
Simple flow:
Accidental Merge
↓
Check Changes
↓
Check CI/CD
↓
Check Environment Impact
↓
Revert if Required
↓
Fix Protection Rules
Main point:
Accidental Merge → Assess impact → Revert if needed → Strengthen controls.

---

29. GitHub Actions or Jenkins pipeline is using an old commit. What will you check?
Definition:
A CI/CD pipeline may build an unexpected revision if the trigger, branch, tag, or checkout configuration is incorrect.
First, I identify exactly which Git reference triggered the pipeline.
Then I verify the commit SHA checked out by the pipeline and compare it with the expected source revision.
I also check workspace state, caching, webhook events, and pipeline configuration if the wrong revision is being built.
Simple meaning:
Pipeline old code build chesthe first which branch/tag/commit trigger ayindo check chestha.
Pipeline actual ga ye commit checkout chesindo verify chestha.
Tarvata webhook, workspace, cache, and pipeline configuration check chestha.
Example:
Suppose a new commit was pushed to main but Jenkins built the previous commit.
I compare the expected commit SHA with the checkout SHA in the Jenkins logs and investigate the trigger or workspace configuration.
Simple flow:
Git Push
↓
Trigger
↓
Pipeline
↓
Checkout
↓
Commit SHA
↓
Compare Expected vs Actual
↓
Fix Configuration

Main point:
Old Commit → Check Trigger → Check Checkout SHA → Check Workspace/Cache → Fix.

---

30. Explain a real-time Git → GitHub → Jenkins → Docker → Kubernetes deployment flow.
Definition:
A typical CI/CD workflow starts with developers managing source code through Git and collaborating through GitHub.
GitHub Pull Requests provide code review and controlled integration into the main branch.
Jenkins can automatically build, test, and package the application after approved changes are merged.
The resulting Docker image is stored in a registry and the exact version is deployed to Kubernetes.
Simple meaning:
Developer feature branch lo code change chestadu and GitHub ki push chestadu.
Pull Request create chesi review and CI checks complete chestham.
Main branch merge ayyaka Jenkins application build and test chesi versioned Docker image create chestundi.
Image registry lo store ayyaka same image version Kubernetes lo deploy chestham.
Example:
Suppose a developer changes the backend application.
Developer creates a feature branch and pushes the code to GitHub.
A Pull Request is created and Jenkins runs build, unit tests, and code-quality checks.
After approval, the change is merged into main.
Jenkins builds a Docker image with a unique version, pushes it to the registry, and deploys that exact image to Kubernetes.
After deployment, the rollout and application health are verified.
Simple flow:
Developer
↓
Git Feature Branch
↓
GitHub Pull Request
↓
Code Review
↓
Jenkins CI
↓
Build + Test
↓
Docker Image
↓
Container Registry
↓
Kubernetes
↓
Deployment
↓
Health Check
↓
Monitor / Rollback
Main point:
Git → GitHub → PR → Jenkins → Build/Test → Docker → Registry → Kubernetes → Verify → Rollback.
ChatGPT is